

SECUREVAULT

Secure Password Management System

PRG800 – Systems Analysis and Design

Milestone 3 – Requirements Analysis

Seneca Polytechnic • PRG800NAA.06353.2264

June 9, 2026

Name	Student #	Email
Felix Akuetteh Torto	168365229	fatorto@myseneca.ca
Akshay Nedumparambil	190635235	anedumparambil-unnik@myseneca.ca
Harmon Anthony Tuazon	165229220	hamtuazon@myseneca.ca
Jean Luc Sita Mbuya	111827242	sjlmbuya@myseneca.ca
Kulsum Timol	112867247	katimol@myseneca.ca
Anju Babu	115640245	ababu35@myseneca.ca

Table of Contents

Table of Contents.....	2
Introduction & Executive Summary	3
Part A — Stakeholder Analysis	4
A.1 Stakeholder Identification	4
A.2 Power-Interest Analysis.....	4
A.2.1 Power-Interest Grid.....	4
A.2.2 Communication Strategies by Quadrant	5
A.3 Context Diagram (Level-0 DFD).....	5
A.3.1 System Boundary Summary	5
A.3.2 Data Flow Table	5
A.3.3 Notes on Diagram Conventions.....	6
Part B — Use Cases.....	7
B.1 Event Analysis	7
B.2 Use Case Diagram Overview	7
B.3 Use Case Narratives.....	8
UC-01 Log In	8
UC-02 Add Credential	9
UC-03 Retrieve Credential	9
UC-04 Store Sensitive Document	10
UC-05 Analyze Password Health & Notify	10
Part C — Domain Class Diagram.....	12
C.1 Class Diagram	12
.....	12
C.2 Class Dictionary.....	12
C.3 Relationships	13
Part D — Non-Functional Requirements.....	14
D.1 Constraints.....	14
D.2 Business Rules.....	14
D.3 Quality Requirements.....	14
References.....	15
Summary.....	16

Introduction & Executive Summary

SecureVault is a web-based, zero-knowledge password management system developed as a group project for PRG800 — Systems Analysis and Design at Seneca Polytechnic. The system enables individual users to store, generate, and manage credentials and sensitive documents within an encrypted vault, protected by AES-256 encryption at rest and multi-factor authentication.

This Milestone 3 report documents the requirements analysis for SecureVault. It is organized into four parts that progress from stakeholder context through functional requirements, domain modelling, and non-functional requirements — following the standard systems development lifecycle.

Part A (Stakeholder Analysis) identifies all parties with an interest in SecureVault, maps their relative power and interest, and defines the system boundary through a Level-0 context diagram. Part B (Use Cases) formalizes the system's functional requirements as an event analysis, a use case diagram, and five fully narrated use case descriptions. Part C (Domain Class Diagram) presents the object model with class attributes, relationships, inheritance, and composition. Part D (Non-Functional Requirements) enumerates the constraints, business rules, and quality attributes — the “ilities” — the system must satisfy.

The team applied industry-standard modelling techniques throughout, including UML 2.5 notation for class and use case diagrams, DFD-0 conventions for the context diagram, and NIST SP 800-63B and ISO/IEC 27001 standards for security controls. All source artefacts are maintained under GitHub version control.

Part A — Stakeholder Analysis

This section presents the complete stakeholder analysis for SecureVault, covering stakeholder identification, power-interest analysis, and the system context diagram.

A.1 Stakeholder Identification

The table below lists all identified stakeholders and their primary interest in SecureVault. Stakeholders who interact directly with the running system are classified as Actors; those who influence or are affected by the project but do not interact at runtime are classified as Stakeholders only.

Stakeholder / Actor	Type	Primary Interest in SecureVault
End User (Individual / Student)	Actor	Primary system user. Needs a secure, easy-to-use platform to store, generate, and manage passwords and sensitive credentials. Interacts directly with every module: vault, generator, health dashboard, 2FA, and file storage.
System Administrator	Actor	Deploys and maintains the SecureVault application. Manages server configuration, database, backups, and security patches. Interacts with deployment scripts, audit logs, and environment settings.
Authentication Service (2FA)	Actor (External)	External system (e.g., TOTP/FIDO2 provider) that issues and validates second-factor tokens. Interacts with the authentication module during login and 2FA enrollment flows.
Email Service	Actor (External)	Third-party SMTP or transactional email provider. Delivers account-verification, password-reset, and security-alert emails to users.
Prof. Miles McDonald (Sponsor)	Stakeholder	Academic client and project sponsor. Evaluates deliverables against PRG800 requirements. Sets milestone deadlines and acceptance criteria but does not interact with the running system.
SecureVault Development Team	Stakeholder	Six-member team responsible for design, development, QA, and documentation. Shape all technical decisions. Accountable for delivering a working prototype on schedule.
Seneca Polytechnic (Institution)	Stakeholder	Provides the academic environment, infrastructure access, and reputational context. Interested in students applying cybersecurity best practices and producing professional-grade work.
Potential Future Enterprise Clients	Stakeholder	Organisations that might adopt SecureVault post-academic phase. Interested in scalability, compliance, and integration capabilities. Currently out of scope.

Note: External systems (Authentication Service and Email Service) are classified as Actors because they exchange data flows with SecureVault at the system boundary, consistent with DFD-0 / context-diagram conventions.

A.2 Power-Interest Analysis

The Power-Interest framework plots each stakeholder by (a) their authority or influence over the project and (b) their degree of stake in its outcome. Quadrant placement drives the recommended communication strategy.

A.2.1 Power-Interest Grid

Vertical axis: Power (authority / influence) | Horizontal axis: Interest (stake in outcome)

	Low Interest	High Interest
HIGH POWER	KEEP SATISFIED Seneca Polytechnic	MANAGE CLOSELY Prof. McDonald Dev Team
LOW POWER	MONITOR Future Enterprise Clients	KEEP INFORMED End User System Admin Auth Service Email Service

A.2.2 Communication Strategies by Quadrant

Stakeholder	Quadrant	Power	Interest	Communication Strategy
Prof. McDonald	Manage Closely	High	High	Milestone reports at each deadline; proactive escalation of scope or timeline risks; formal sign-off on deliverables.
Dev Team	Manage Closely	High	High	Daily Scrum stand-ups; GitHub project board; sprint retrospectives; shared communication channel.
Seneca Polytechnic	Keep Satisfied	High	Low	Ensure academic integrity and professional documentation standards; align with institutional cybersecurity policy.
End User	Keep Informed	Medium	High	UX walkthroughs and usability testing during Sprints 3–4; feedback surveys; release notes on each prototype iteration.
System Admin	Keep Informed	Medium	Medium	Deployment runbooks and environment specs shared at Sprint 6; admin console documentation kept current in GitHub.
Auth / Email Services	Keep Informed	Low	Medium	Integration contracts (API specs, SLAs) defined in Sprint 2; change notifications via API versioning; no ongoing dialogue needed.
Future Clients	Monitor	Low	Low	Note scalability and enterprise-relevant features in design decisions; no active communication during academic phase.

A.3 Context Diagram (Level-0 DFD)

A context diagram (Level-0 DFD) places SecureVault as a single process at the centre. All external entities surround it; labelled arrows show the data flows crossing the system boundary in each direction.

A.3.1 System Boundary Summary

- Central process: SecureVault – Secure Password Management System
- External entities: End User, System Administrator, 2FA Auth Service, Email Service, Database, Prof. McDonald
- Data flows are bidirectional for all entities — inputs to the system and outputs from the system are both labelled

[Context Diagram Figure — see attached diagram artifact]

A.3.2 Data Flow Table

→ *Inbound (data entering the system boundary)* | ← *Outbound (data leaving the system boundary)*

External Entity	Direction	Data Flows
End User	→ SecureVault	Login credentials, master password, vault commands (add / edit / delete), password generator requests, file upload
End User	← SecureVault	Decrypted credentials on request, generated passwords, password health scores, security alerts, audit log view
System Administrator	→ SecureVault	Deployment commands, configuration parameters, environment variables, maintenance scripts
System Administrator	← SecureVault	Audit logs, system health reports, error logs, user activity summaries
2FA Auth Service	→ SecureVault	OTP validation result (success / failure), token metadata
2FA Auth Service	← SecureVault	OTP verification request (user ID, token presented)
Email Service	→ SecureVault	Delivery status (sent / failed / bounced)
Email Service	← SecureVault	Email send request (recipient, type: verification / reset / alert, content payload)
Database	→ SecureVault	Encrypted credential records, user profiles, audit entries, session tokens
Database	← SecureVault	Encrypted write requests, query parameters, update / delete operations
Prof. McDonald	→ SecureVault	Requirements, constraints, acceptance criteria, milestone feedback
Prof. McDonald	← SecureVault	Milestone reports, working prototype demonstrations, final submission documentation

A.3.3 Notes on Diagram Conventions

- The Database is treated as an external entity at context level, consistent with standard DFD-0 conventions.
- External systems (2FA Auth Service, Email Service) appear as actors because they exchange data flows at the system boundary even though they are not human users.
- A Level-1 DFD would decompose SecureVault into its internal processes: authentication, vault management, password generator, health analyser, file storage, and audit logger.

Part B — Use Cases

This section defines the functional requirements of SecureVault through structured event analysis, a use case diagram overview, and detailed use case narratives.

B.1 Event Analysis

No.	Event Type	Event (Trigger)	Use Case Triggered	Result / Output
1	External	User submits login and 2FA code	Log In	Authenticated session opened
2	External	User saves a new credential	Add Credential	Credential encrypted and stored
3	External	User opens a stored credential	Retrieve Credential	Credential decrypted and shown
4	External	User requests a new password	Generate Password	Strong password returned
5	External	User uploads a sensitive document	Store Document	Document encrypted and stored
6	External	User opens a stored document	Retrieve Document	Document decrypted and shown
7	Temporal	Idle timeout is reached	Lock Vault	Session ended, vault locked
8	State	A saved password is weak	Notify User (Weak)	Weak-password alert shown
9	State	A password repeats one used recently	Notify User (Reused)	Reuse alert shown

B.2 Use Case Diagram Overview

The use case diagram shows the SecureVault boundary, the User actor, the core use cases, and the supporting use cases linked by «include» and «extend» relationships.

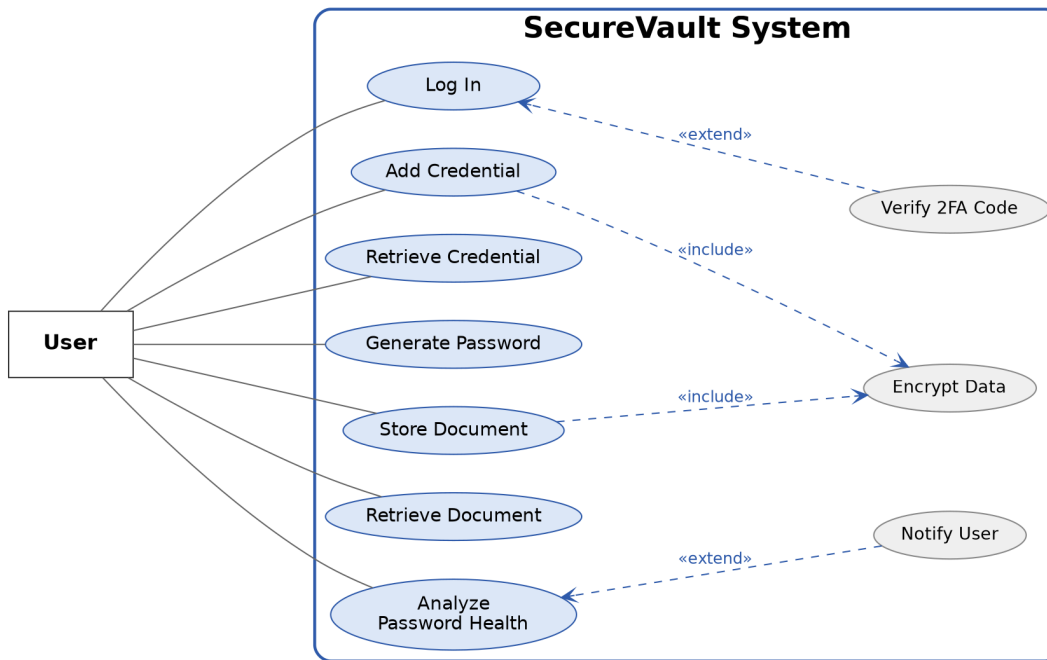


Figure B.1: SecureVault Use Case Diagram

Relationships:

- Verify 2FA Code «extend» Log In (runs only when 2FA is enabled)
- Notify User «extend» Analyze Password Health (fires only when a risk is found)
- Add Credential and Store Document «include» Encrypt Data (encryption is always part of saving)

B.3 Use Case Narratives

The following five narratives cover the system's core use cases. Each team member is responsible for at least two fully defined narratives.

UC-01 Log In

Use Case Name	Log In
Description	A registered user signs in with a master password and a second factor to open the encrypted vault.
Trigger	User opens the app and submits login details.
Preconditions	User has a registered account. 2FA is set up.
Postconditions	A secure session starts and the vault is unlocked. The event is logged.
Stakeholders & Actors	Actor: User. Supporting: Email / Authenticator Service. Stakeholders: User, project team.
Exceptions	Wrong password or code → access denied. Five failures → account locked 15 minutes.

Interaction Dialog:

Actor	System
1. Enters email and master password.	2. Verifies the password hash.
	3. Requests the 2FA code («extend»: Verify 2FA Code).
4. Enters the 2FA code.	5. Validates the code and starts the session.
	6. Decrypts and shows the vault; records the login.

UC-02 Add Credential

Use Case Name	Add Credential
Description	A user saves a new login (site, username, password) into the encrypted vault.
Trigger	User chooses to add a credential.
Preconditions	User is logged in and the vault is unlocked.
Postconditions	The credential is encrypted, stored, and written to the audit log.
Stakeholders & Actors	Actor: User. Stakeholders: User, project team.
Exceptions	Required field empty → save blocked. Encryption fails → entry not saved, error shown.

Interaction Dialog:

Actor	System
1. Opens the add-credential form.	2. Displays the empty form.
3. Enters site, username, password.	4. Validates the fields.
5. Saves the entry.	6. Encrypts the data («include»: Encrypt Data).
	7. Stores it and logs the action («include»: Record Audit Entry).

UC-03 Retrieve Credential

Use Case Name	Retrieve Credential
Description	A user opens a stored credential and views or copies the password.
Trigger	User selects an entry from the vault.
Preconditions	User is logged in; the entry exists.
Postconditions	The decrypted credential is shown; the access is logged.
Stakeholders & Actors	Actor: User. Stakeholders: User, project team.
Exceptions	Entry not found → message shown. Decryption fails → access denied.

Interaction Dialog:

Actor	System
-------	--------

1. Searches or selects an entry.	2. Locates the encrypted record.
	3. Decrypts the data («include»: Decrypt Data).
4. Views or copies the password.	5. Shows the entry and logs the access.

UC-04 Store Sensitive Document

Use Case Name	Store Sensitive Document
Description	A user uploads a sensitive document (for example an ID scan or recovery codes); the system encrypts and stores it in the vault.
Trigger	User chooses to upload a document.
Preconditions	User is logged in and the vault is unlocked.
Postconditions	The document is encrypted, stored, and written to the audit log.
Stakeholders & Actors	Actor: User. Stakeholders: User, project team.
Exceptions	Wrong file type or oversized file → upload blocked. Encryption fails → file not saved.

Interaction Dialog:

Actor	System
1. Selects a file to upload.	2. Checks the file type and size.
3. Confirms the upload.	4. Encrypts the file («include»: Encrypt Data).
	5. Stores it and logs the action.

UC-05 Analyze Password Health & Notify

Use Case Name	Analyze Password Health & Notify
Description	The system checks stored passwords for weak strength and recent reuse, and notifies the user of any risk it finds.
Trigger	User opens the health report, a saved password changes, or a scheduled scan runs.
Preconditions	User is logged in and the vault has at least one entry.
Postconditions	A health report is shown and the user is alerted to weak or reused passwords.
Stakeholders & Actors	Actor: User. Stakeholders: User, project team.
Exceptions	Empty vault → the report shows no entries.

Interaction Dialog:

Actor	System
1. Opens the health report.	2. Scans all stored passwords.
	3. Scores strength and checks for reuse in the last few weeks.

	4. Shows the report.
	5. Alerts the user to each risk («extend»: Notify User).

Part C — Domain Class Diagram

This section presents the domain model for SecureVault, capturing key objects, their attributes, and their relationships. Visibility is shown with – for private and + for public. The model is drawn from the Part B use cases and the Part D business rules.

C.1 Class Diagram

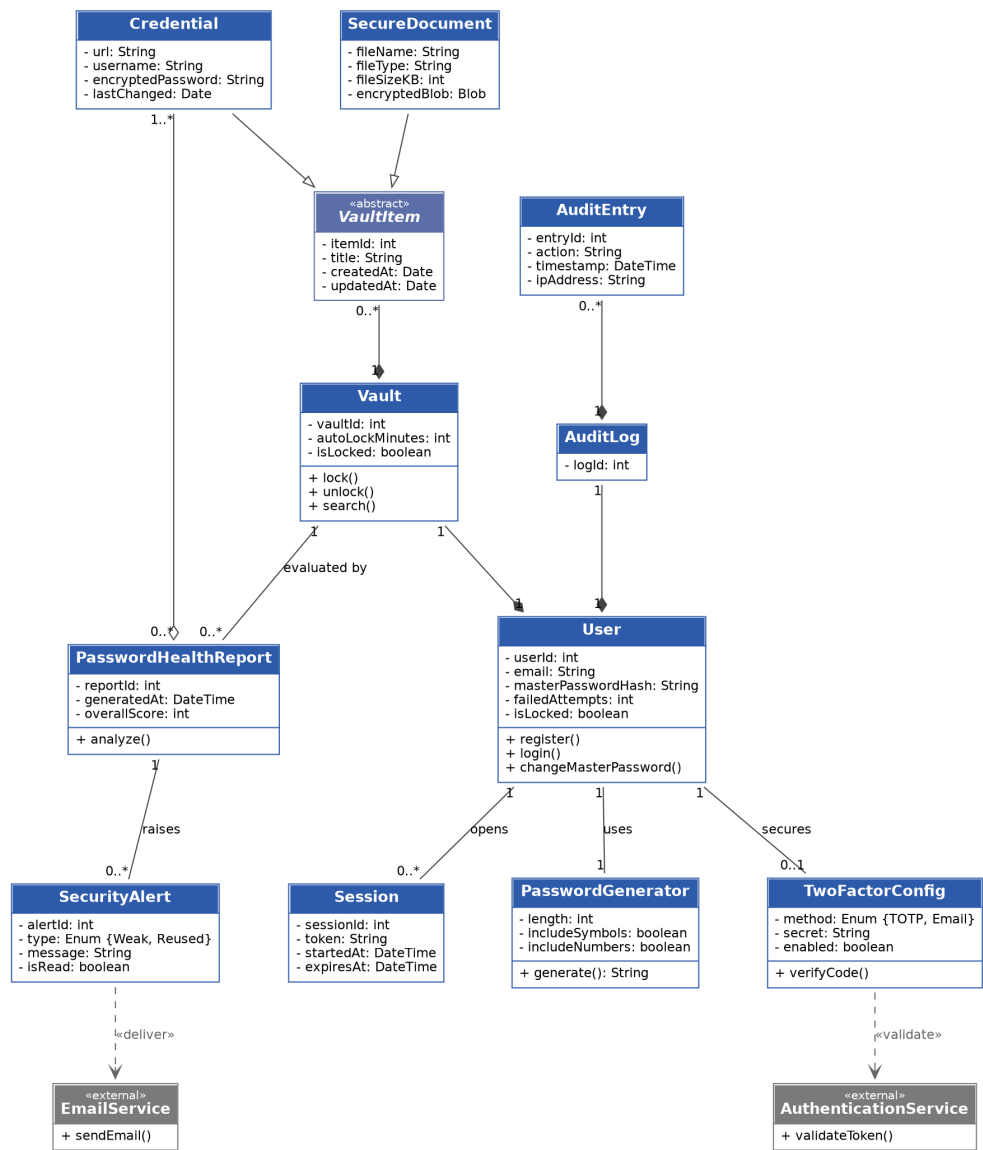


Figure C.1: SecureVault Domain Class Diagram

C.2 Class Dictionary

Class	Purpose / Key Data
User	Account owner; holds login identity and the master-password hash.
Vault	The user's encrypted container for all stored items (one per user).
VaultItem «abstract»	Base type for anything stored in the vault.

Credential	A stored login: site, username, and encrypted password.
SecureDocument	An encrypted sensitive file, e.g. an ID scan or recovery codes.
TwoFactorConfig	A user's second-factor method and secret.
Session	An active, time-limited login session.
AuditLog	Append-only record of a user's actions.
AuditEntry	A single logged action with timestamp and IP address.
PasswordHealthReport	Result of scanning a vault for weak or reused passwords.
SecurityAlert	A weak- or reused-password notification raised for the user.
PasswordGenerator	Service that builds strong passwords to chosen rules.
AuthenticationService «external»	Third-party service that validates 2FA tokens.
EmailService «external»	Third-party service that delivers verification, reset, and alert emails to users.

C.3 Relationships

- Inheritance: Credential and SecureDocument are types of VaultItem and share its common fields.
- Composition: a User owns exactly one Vault and one AuditLog; a Vault is made up of its VaultItems; an AuditLog is made up of its AuditEntries. A part cannot exist without its whole.
- Aggregation: a PasswordHealthReport groups the Credentials it evaluates, but those Credentials exist independently of any report.
- Associations: a User has one optional TwoFactorConfig, opens many Sessions, and uses the PasswordGenerator; a Vault is evaluated by many PasswordHealthReports; a report raises many SecurityAlerts.
- Dependencies: TwoFactorConfig relies on the external AuthenticationService to validate tokens; SecurityAlert relies on the external EmailService to deliver messages.

Part D — Non-Functional Requirements

This section documents the constraints, business rules, and quality attributes that govern SecureVault.

D.1 Constraints

- Must be built within the academic term, with limited team hours.
- Uses free, open-source tools only: Node.js, Express, MySQL or MongoDB, GitHub.
- No specialized hardware is needed for build, test, or demo.
- Web-based and single-user in scope; no enterprise or commercial deployment.
- Small team with mixed skill levels.

D.2 Business Rules

- The master password is never stored; only its hash is kept.
- The master password must be at least 12 characters and mix character types.
- All vault data is encrypted at rest (AES-256) and sent over TLS.
- 2FA is required when logging in from a new device.
- The vault auto-locks after 10 minutes of inactivity.
- A user can access only their own vault.
- The audit log is append-only and cannot be edited by users.
- A password is flagged as weak if it is too short or low in character variety.
- A password is flagged when it repeats another entry or one used in the last 30 days.
- Stored documents are encrypted at rest the same way as passwords (AES-256).
- Uploaded documents are limited by type and size (PDF or image, up to 10 MB).

D.3 Quality Requirements

Key system qualities (the “ilities”) the design must meet:

Quality	Requirement
Security	AES-256 at rest, TLS in transit, hashed master password, and 2FA.
Confidentiality	Zero-knowledge design; the server never sees plaintext passwords.
Integrity	Encryption and an append-only audit log prevent silent tampering.
Reliability	Errors are handled gracefully with no loss of stored data.
Availability	Target 99% uptime during use and demos.
Usability	Clean interface; core tasks done in three clicks or fewer.
Performance	Vault unlock and search complete in under two seconds.
Maintainability	Modular, documented code under GitHub version control.
Scalability	Architecture supports future multi-user expansion without redesign.
Serviceability	Errors are logged and surfaced to the administrator with clear diagnostics.

References

- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- International Organization for Standardization. (2022). ISO/IEC 27001:2022 — Information security management systems. ISO. <https://www.iso.org/standard/27001>
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The unified software development process. Addison-Wesley.
- Larman, C. (2004). Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development (3rd ed.). Prentice Hall.
- Mendelow, A. L. (1991). Environmental scanning: The impact of the stakeholder concept. Proceedings of the 2nd International Conference on Information Systems.
- National Institute of Standards and Technology. (2020). NIST SP 800-63B: Digital identity guidelines — Authentication and lifecycle management. NIST. <https://pages.nist.gov/800-63-3/sp800-63b.html>
- National Institute of Standards and Technology. (2001). FIPS 197: Advanced Encryption Standard. NIST. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- Object Management Group. (2017). Unified Modeling Language specification version 2.5.1. OMG. <https://www.omg.org/spec/UML/2.5.1>
- Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.
- Yourdon, E., & DeMarco, T. (1979). Structured design: Fundamentals of a discipline of computer program and systems design. Prentice Hall.

Summary

This report demonstrates that SecureVault is a well-scoped, security-focused system with a clearly defined set of stakeholders, functional requirements, and structural constraints. The stakeholder analysis confirmed that the End User and Development Team are the primary high-power, high-interest parties, with Prof. McDonald serving as the academic sponsor who requires close engagement at each milestone.

The use case analysis identified nine triggering events — spanning external user actions, temporal auto-lock, and state-based health alerts — and produced five detailed narratives covering login with 2FA, credential management, document storage, and password health analysis. These narratives form the functional baseline from which all design and implementation work will proceed.

The domain class diagram established fourteen classes with clear inheritance (Credential and SecureDocument as subtypes of VaultItem), strong composition relationships (User → Vault → VaultItems), and dependency links to external services.

The non-functional requirements articulate a zero-knowledge security posture — master passwords are never stored in plaintext, all vault data is AES-256 encrypted at rest, and TLS protects all data in transit. Performance, usability, maintainability, and availability targets are defined and measurable. Together, these artifacts provide a solid foundation for the design and implementation milestones that follow.